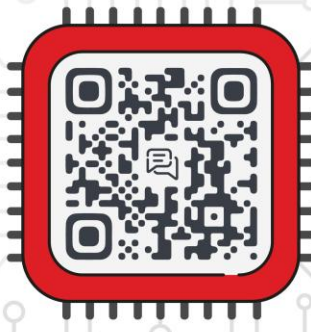




4



15



1800 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

التوارد (1) Pipelining (1)

عملي مشترك



RB Informatics; 10/11/2025

بنیان الحواسيب 2

مقدمة: مرحبا أصدقائي، جاهزين نكمل مشوارنا؟

اليوم بالمحاضرة الرابعة رح نفهم أفكار بتخلي كل شي تعلمناه قبل ياخذ معنى جديد، شدوا الهمة وخليكم مستمرين

السؤال: ارسم مخطط الحالات للألة منتهية الحالات التي قمنا باستنتاج معادلاتها سابقا مع ترميز كل الحالات؟

الحل:

لدينا 17 حالة - > نحتاج 5 بت لترميزهم

تذكرة:

الحالات هي:

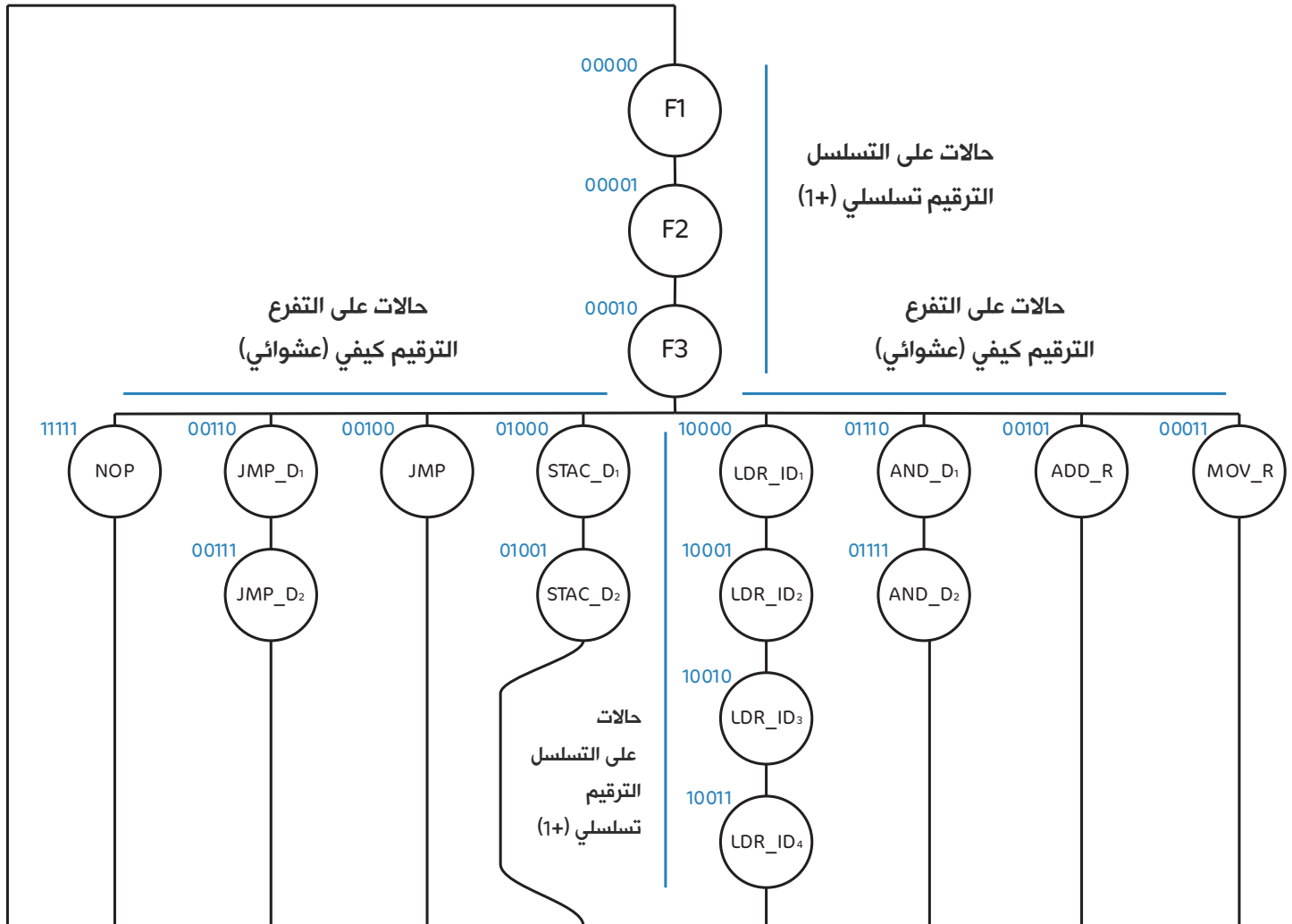
$F1 : AR < - AC$
 $F2 : DR < - M[AR] ; PC < - PC + 1$
 $F3 : IR < - DR(op) ; AR < - DR(x)$
 $MOV_R : R < - AC$
 $ADD_R : AC < - AC + R$
 $AND_D_1 : DR < - M[AR]$
 $AND_D_2 : AC < - DR \text{ and } AC$
 $LDR_ID_1 : DR < - M[AR]$
 $LDR_ID_2 : AR < - DR$
 $LDR_ID_3 : DR < - M[AR]$
 $LDR_ID_4 : R < - DR$
 $STAC_D_1 : DR < - AC$
 $STAC_D_2 : M[AR] < - DR$
 $JMP\ x : PC < - DR(x)$
 $JMP_D_1 : DR < - M[AR]$
 $JMP_D_2 : PC < - DR(x)$
 $NOP : nothing$

$F1 \rightarrow 0$

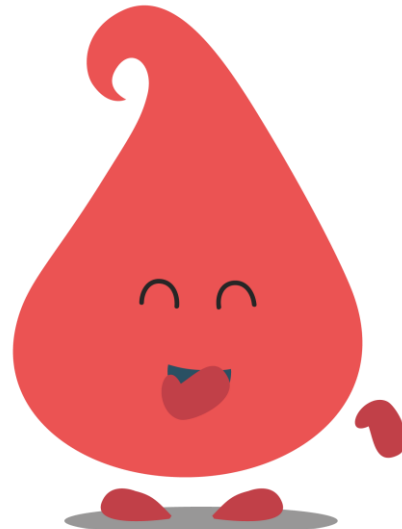
$F2 \rightarrow 1$

$F3 \rightarrow 2$

بعد $F3$ تستطيع إعطاء ترميز للحالات بشكل عشوائي بشرط أن كل حالة لديها تفرعات تليها تكون معها على نفس التسلسل وعند الانتهاء من كل حالة سنعود للحالة $F1$



Smile
يامبوزم

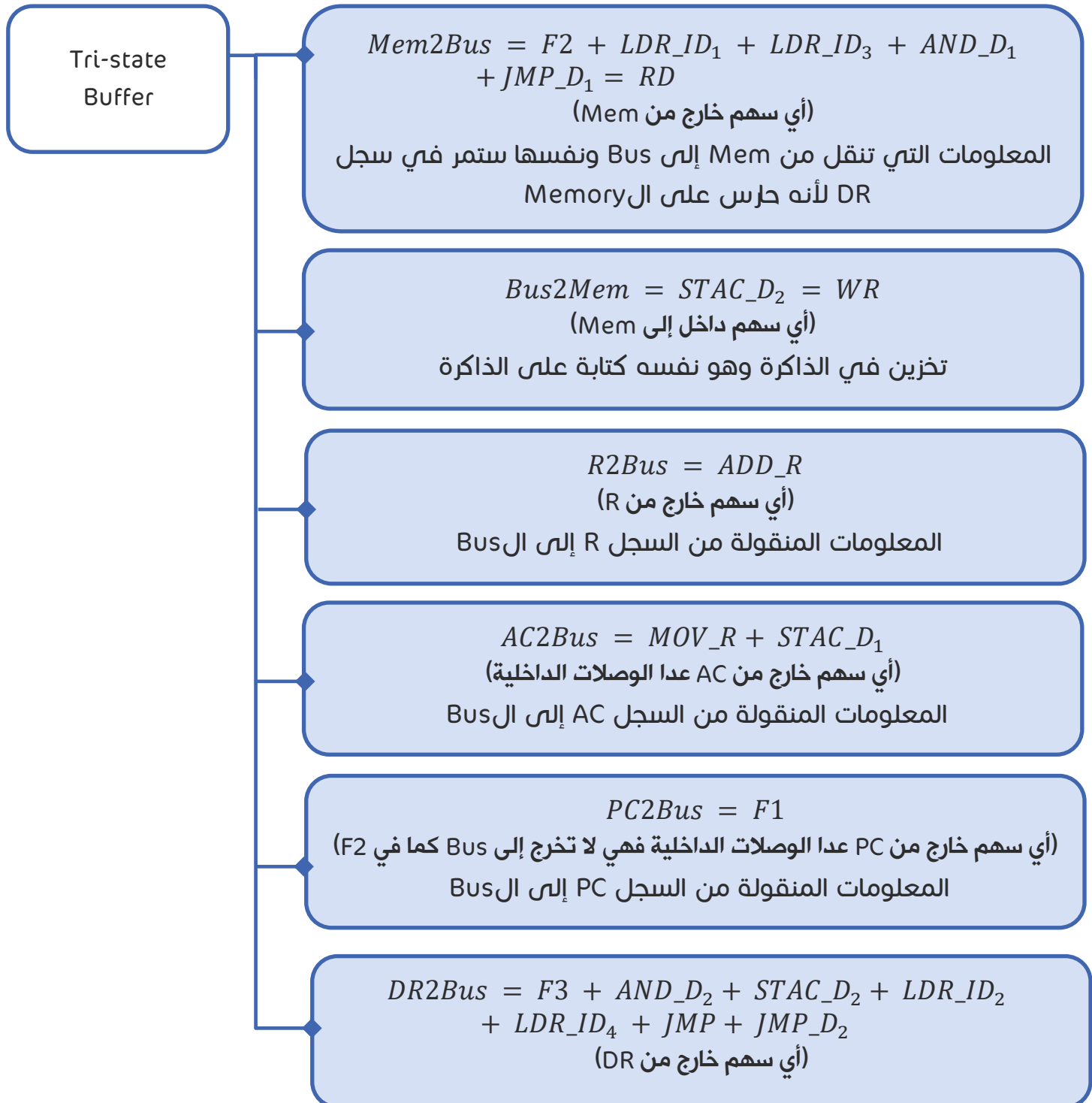


سؤال: ارسم التصميم العتادي لوحدة التحكم (Hard Wired CU) واستنتج معادلات إشارات التحكم بالعداد (INC - Load - Reset)

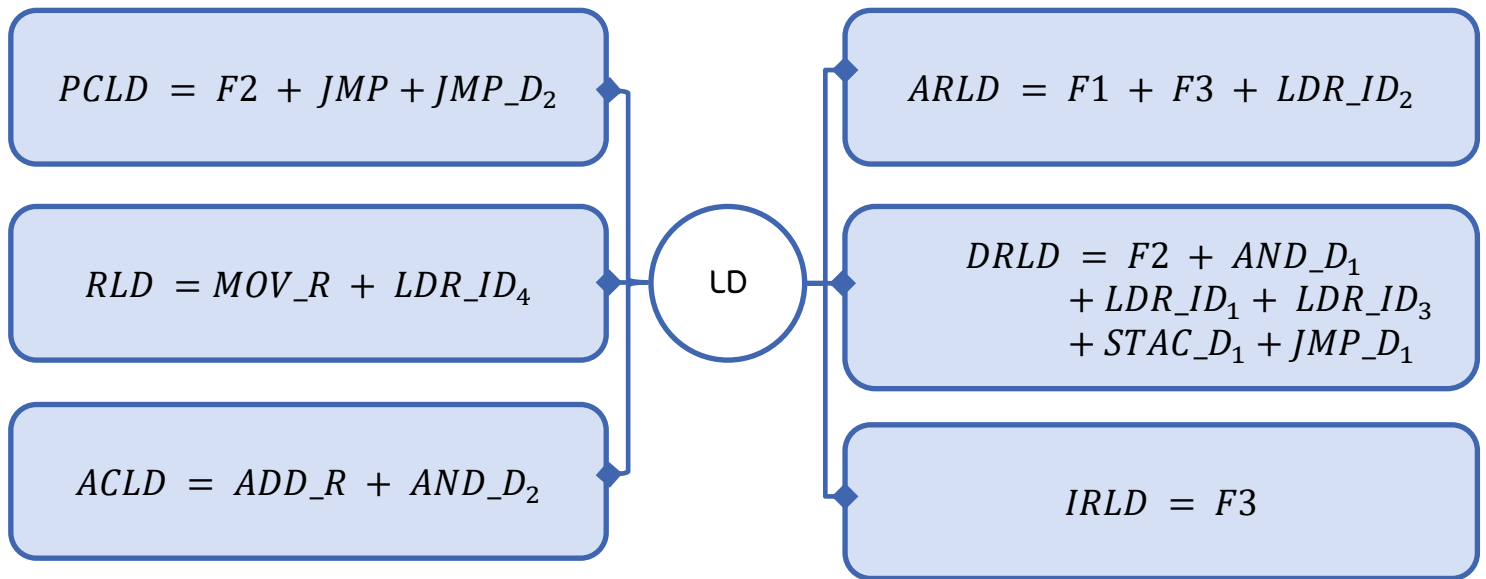
تذكرة:

معادلات التحكم:

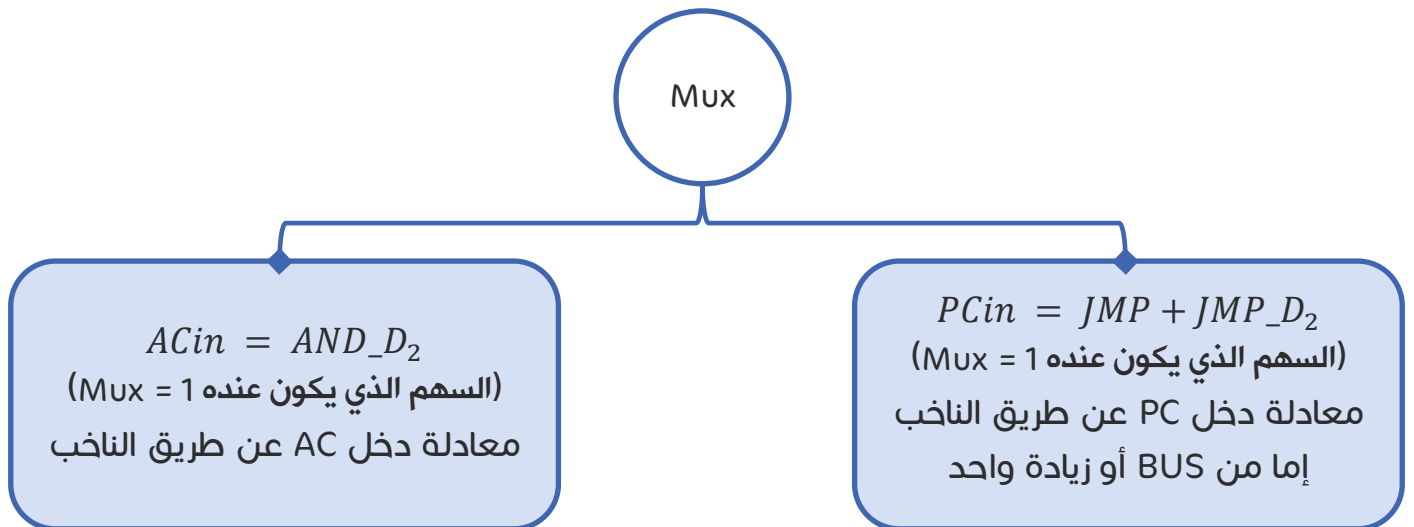
1. معادلات Buses



2. معادلات Load

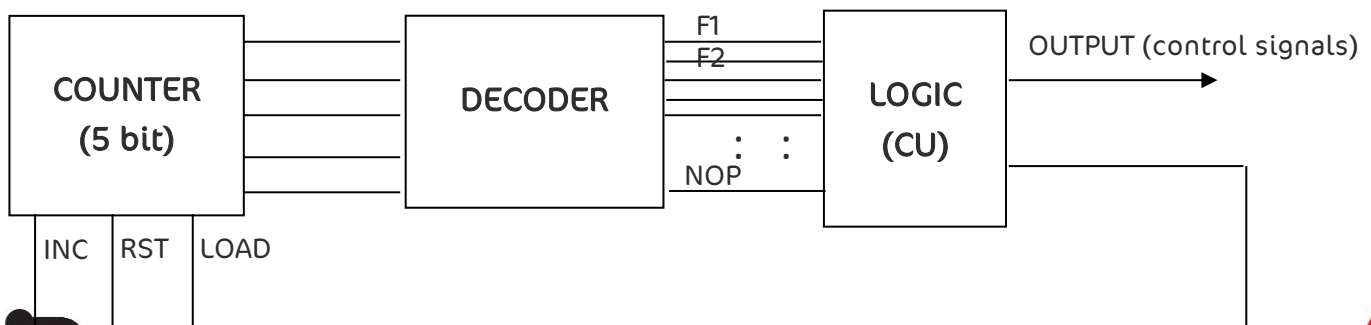


3. معادلات Mux



الحل:

في الحالات الطبيعية كنا نستخدم Flip-Flop لتنفيذ عملية واحدة أو عدد قليل من العمليات لكن لدينا عدد كبير من العمليات لذلك سنستخدم Counter (عداد) ثم نريد مفكك ترميز لمعرفة الخطوة التالية ثم إلى CU



■ INC

يزيد واحد

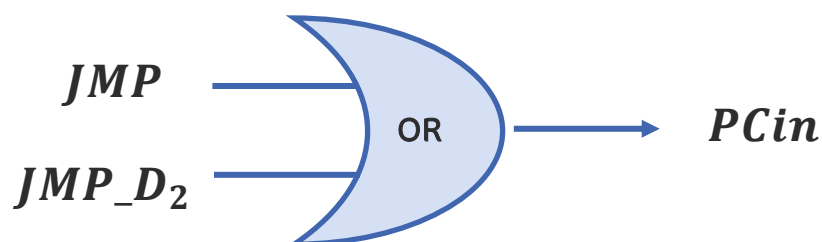
■ RST

يعيد لأول حالة

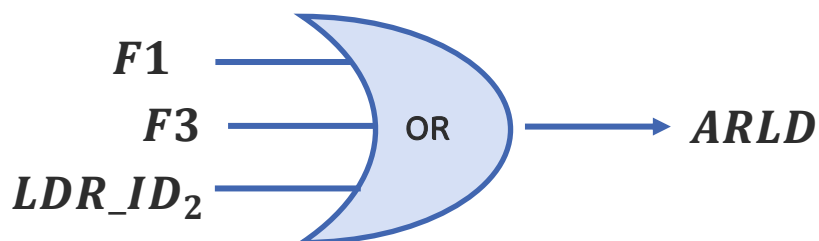
■ LAOD

يحمل قيمة للعداد حسب (OP)

مثلاً:

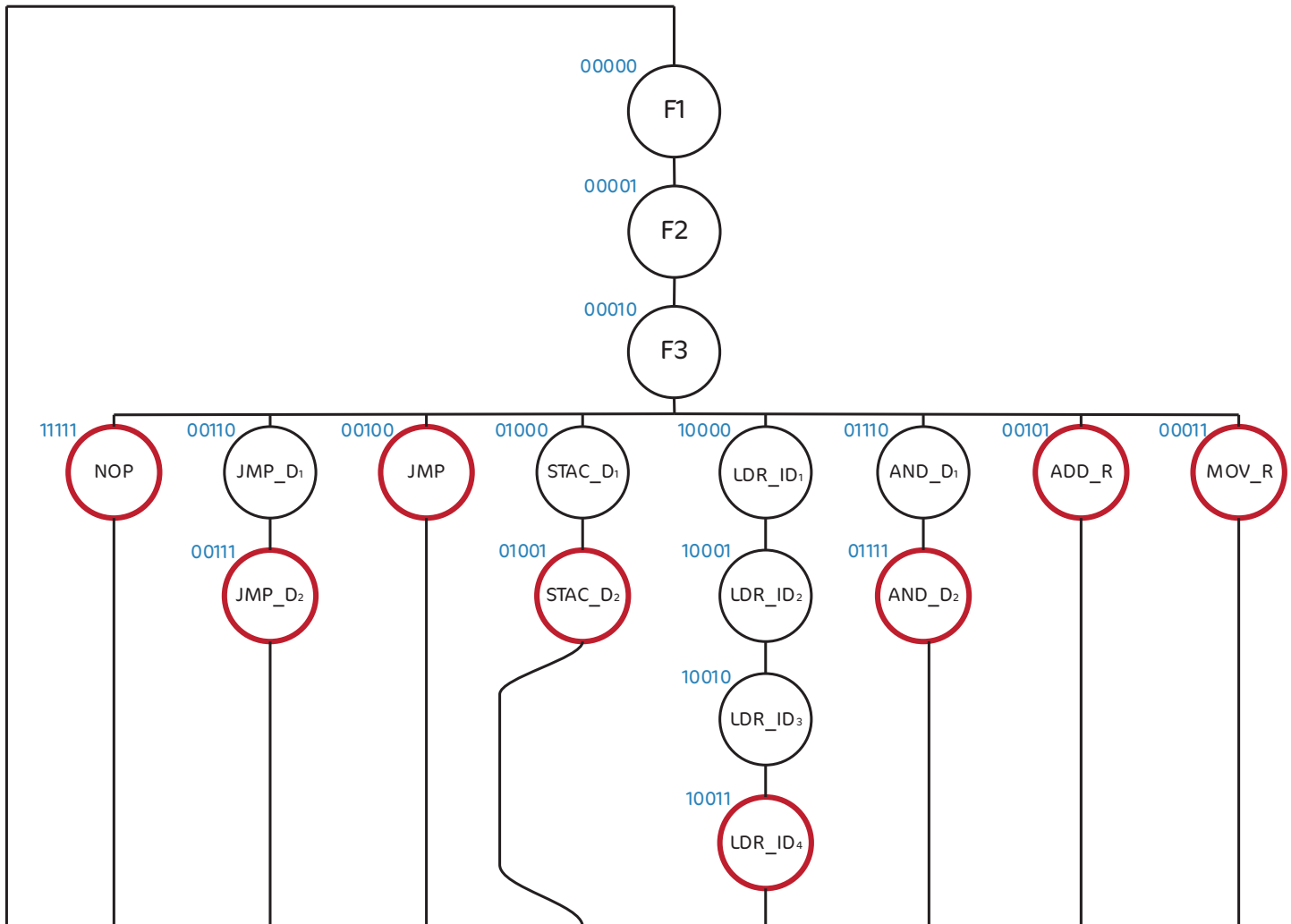


$F1 \longrightarrow PC2BUS$



معادلات إشارات التحكم:

■ نحصل على الإشارة RESET أو CLEAR بأخذ الحالات النهائية لكل تعليمة:

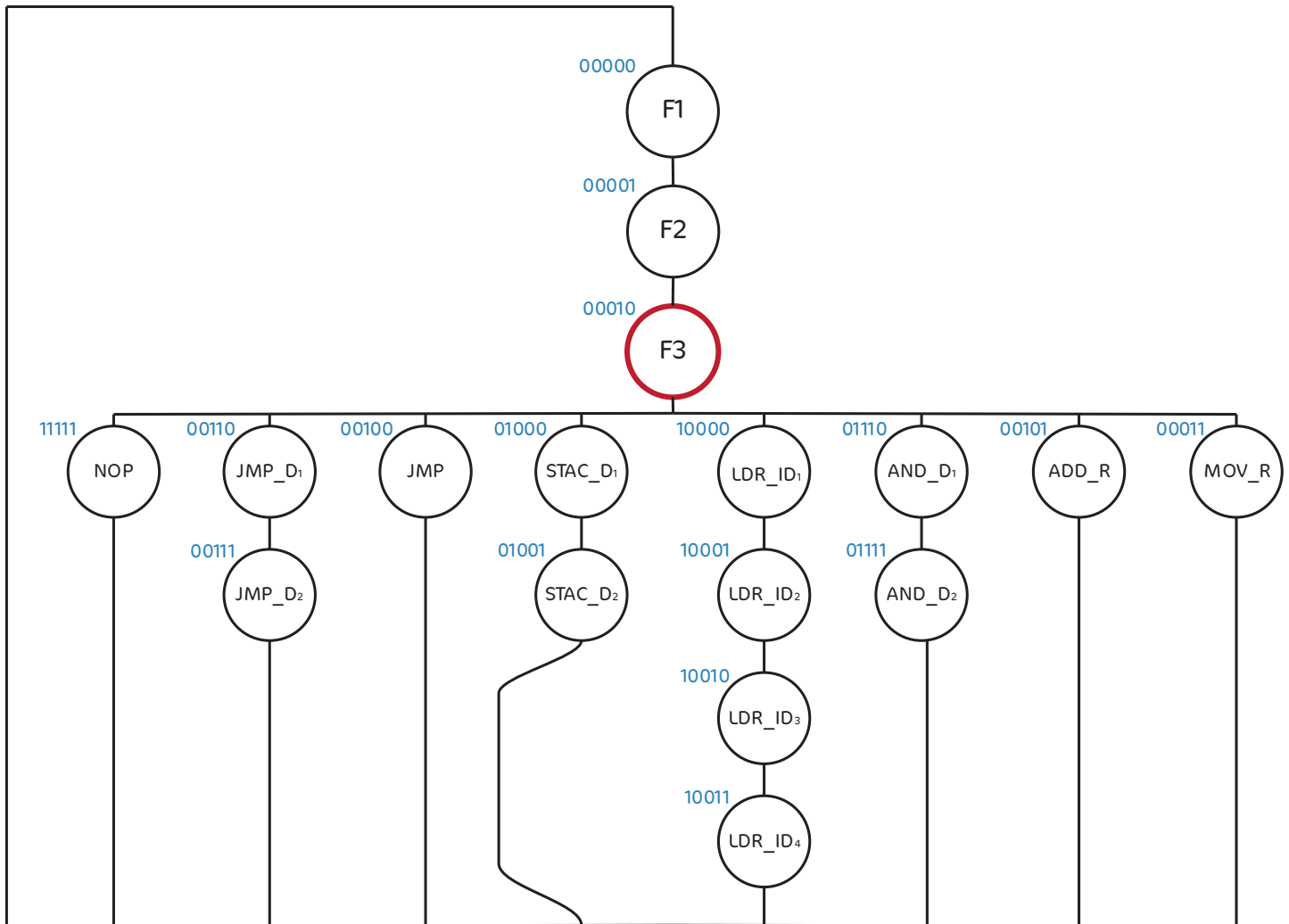


$$RST = NOP + JMP_{D_2} + JMP + STAC_{D_2} + LDR_{ID_4} + AND_{D_2} + ADD_R + MOV_R$$

Not every closed
door is looked,
Push!!

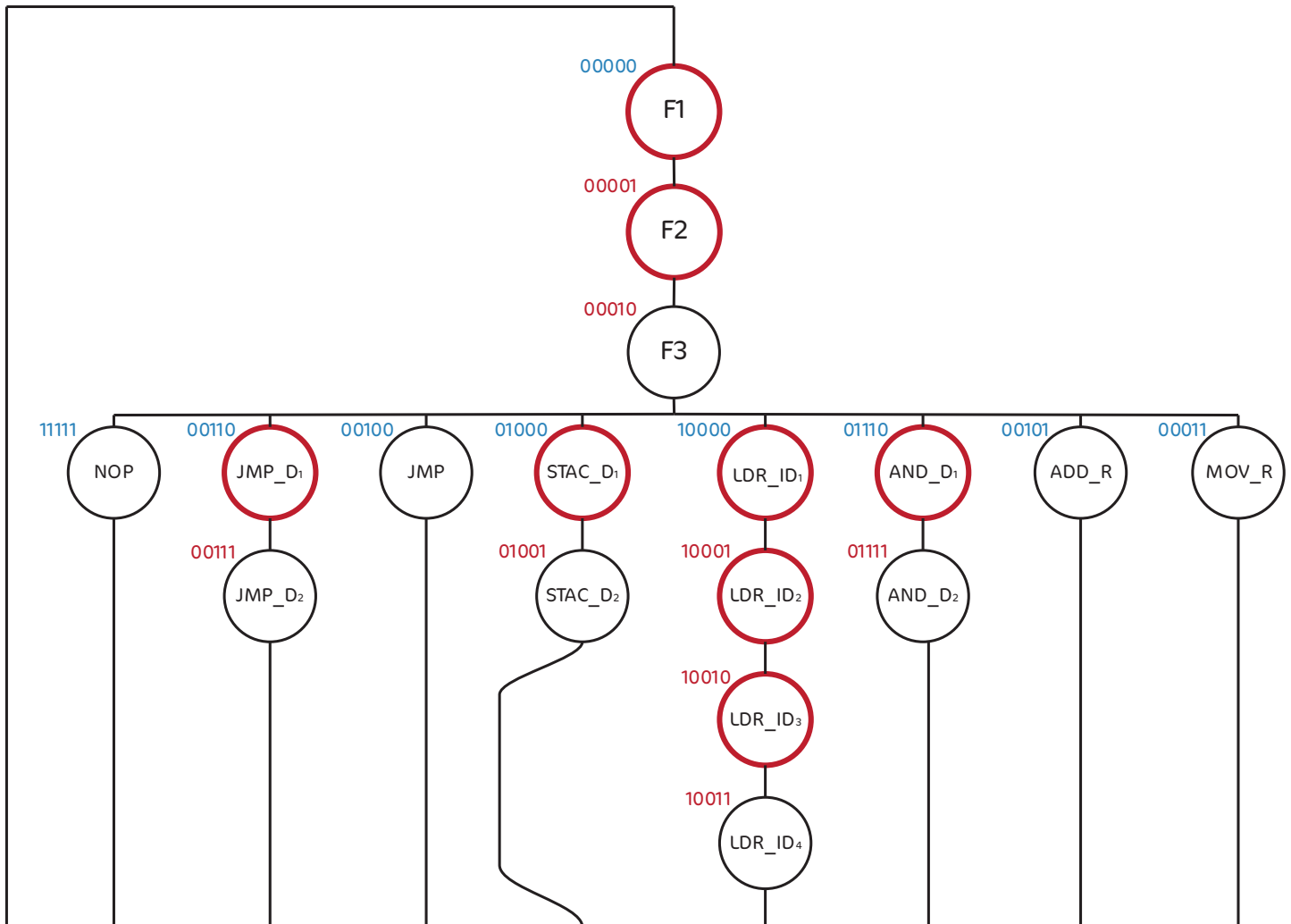


■ نحصل على إشارة التحميل LOAD من مكان وجود التفرع في مخطط الحالات المنتهية، وفي حالتنا فإن:



$$LOAD = F3$$

■ نحصل على إشارة زيادة العداد INC من الحالات التي يزيد فيها العداد بمقدار 1 للانتقال إلى الحالة المستقبلية، وهي جميع الحالات ماعدا الحالات النهائية ومكان وجود التفرع، فهي إذاً:



$$INC = F1 + F2 + JMP_D_1 + STAC_D_1 + LDR_ID_1 + LDR_ID_2 + LDR_ID_3 + AND_D_1$$

خسرت معركة؟؟
ولكن لن تخسر الحرب!



تصميم وحدة التحكم باستخدام البرمجة الصغيرة

Micro-Programmed CU

نستعاض عن تصميم وحدة التحكم بدارات منطقية معقدة لإنتاج إشارات التحكم لكل تعليمة باستخدام ذاكرة خاصة اسمها Micro Memory تحتوي على تعليمات صغيرة (Micro Instructions) وكل واحدة تحتوي على إشارات التحكم المناسبة

1. العملية الصغيرة Micro Operation:

هي العملية الحقيقية التي تحصل داخل المعالج مثل نقل المعلومات من سجل إلى سجل أو من الذاكرة إلى سجل
مثلاً:

$$M \rightarrow DR$$

2. التعليمة الصغيرة:

مكونة من عملية صغيرة واحدة أو أكثر.

مثلاً:

المرحلة F1

3. البرنامج الصغير Micro Program:

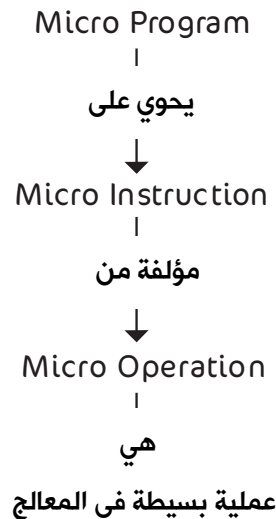
هو تسلسل التعليمات الصغيرة التي تنفذ تعليمة واحدة

مثلاً:

تعليمة ADD تحتاج إلى أربع تعليمات صغيرة

معلومة:

كل التعليمات الصغيرة تخزن في الذاكرة الصغيرة Micro Memory وتسمى كلمات التحكم Control Words لأنها تحدد إشارات التحكم بالضبط



خلاصة

التصميم البرمجي (البرمجة الصغيرة)

كل تعليمة صغيرة داخل الذاكرة الصغيرة تتكون من ثلاث حقول رئيسية

1. حقل الاختبار (Select):

- يتألف من بت واحد
 - يقرر من أين سيأخذ التعليمة
 - إذا كانت قيمته 0 - < العنوان التالي موجود داخل نفس التعليمة
 - إذا كانت قيمته 1 - < العنوان التالي موجود داخل سجل IR
- هذا يعني أنه يتحكم في مسار التنفيذ

2. حقل العنوان (Address):

- هو العنوان الذي يذهب إليه النظام لجلب التعليمة الصغيرة التالية
- مثلاً لو الحقل فيه القيمة 1010 فالتعليمة التالية موجودة بالعنوان 1010

3. حقل التحكم أو العملية الصغيرة (μ OP / Control):

- يحتوي على إشارات التحكم التي فعلياً تتحكم بمكونات وحدة المعالجة مثل (ALU, Buses, Registers)
- محتواها يتغير حسب طريقة الترميز

أنواع الترميز:

1. الترميز المباشر (Direct Coding):

- أبسط نوع
- كل إشارة تحكم لها بت مستقل
- إذا كان لديك 10 إشارات تحكم تحتاج 10 بتات
- سهلة وسريعة لكنها تحتاج مساحة كبيرة في الذاكرة

2. الترميز الأفقي (Horizontal Coding):

- كل عملية صغيرة لها بت منفصل
- يسمح بتفعيل أكثر من عملية صغيرة مع بعض، أي عمليات متوازية داخل المعالج
- أداء وسرعة عالية لكن تحتاج حجم كبير

3. الترميز الشاقولي (Vertical Coding):

- يقوم بترميز العمليات الصغيرة ثنائياً.

مثلاً:

0001 - < نقل

0010 - < جمع

٤ تمرين 1:

ما هو حجم ذاكرة التحكم الصغيرة اللازمة من أجل المسألة السابقة بفرض استخدام الترميز المباشر؟

معلومة:

- عدد أسطر الذاكرة الصغيرة = عدد التعليمات الصغيرة
= عدد الكلمات = عدد الحالات

- حجم الذاكرة الصغيرة = عدد التعليمات الصغيرة
× (عدد إشارات التحكم + عدد البتات اللازمة
لترميزها + بت التحكم)

الحل:

عدد أسطر الذاكرة الصغيرة = 17 سطر لأنه يوجد لدينا 17 حالة ونحتاج إلى 5 بت لترميزهم وبالتالي:

حجم الذاكرة الصغيرة = $17 \times (14 + 5 + 1) = 340$ بت

SELECT	ADDRESS	CONTROL FIELD													
		ARLD	DRLD	IRLD	PCLD	RLD	ACLD	M2BUS	BUS2M	DR2BUS	R2BUS	AC2BUS	PC2BUS	PCIN	ACin
F1	0	00001													
F2	0	00010													
F3	1	XXXXX													
NOP	0	00000													

شرح الجدول:

1. حقل Select

- نضع في حقل Select القيمة 0 عندما نكون نعلم ماذا سينفذ بعدها
مثلاً عند F1 نضع 0 لأننا نعلم أنه سينفذ F2
- نضع في حقل Select القيمة 1 عندما نكون لا نعلم ماذا سينفذ بعدها أي لدينا تفرع
مثلاً عند F3 نضع 1 لأننا لا نعلم ما التعليمة التالية ستكون لدينا عدة خيارات

2. حقل Address

- نضع ترميز التعليمة التي أعرفها عندما أكون أعرفها
- إذا كنت لا أعرف التعليمة أضع الترميز على الشكل xxxxx

3. حقل Control Field

نضع إشارات التحكم على شكل مجموعات ونعود للمعادلات السابقة عند الإشارة التي تكون موجودة في معادلة التعليمة
نضع 1 وباقي أصفار

تمرين 2:

ما هو حجم ذاكرة التحكم الصغيرة اللازمة من أجل المسألة السابقة بفرض استخدام الترميز الأفقي؟

الحل:

الاختلاف عن الطلب السابق بحجم حقل التحكم Control Field حيث فيه بت لكل عملية صغيرة

نضع لائحة بتسميات للعمليات الصغيرة بدون تكرار

العملية الصغيرة	التسمية
$F1: AR < - AC$	ARAC
$F2: DR < - M[AR]$	DRM
$F2: PC < - PC + 1$	INCPC
$F3: IR < - DR(op)$	IRDR
$F3: AR < - DR(x)$	ARDR
$MOV_R: R < - AC$	RAC
$ADD_R: AC < - AC + R$	ADDR
$AND_{D_1}: DR < - M[AR]$	العملية موجودة مسبقاً
$AND_{D_2}: AC < - DR \text{ and } AC$	AND
$LDR_{ID_1}: DR < - M[AR]$	العملية موجودة مسبقاً
$LDR_{ID_2}: AR < - DR$	العملية موجودة مسبقاً
$LDR_{ID_3}: DR < - M[AR]$	العملية موجودة مسبقاً
$LDR_{ID_4}: R < - DR$	RDR
$STAC_{D_1}: DR < - AC$	DRAC

$STAC_D_2: M[AR] < - DR$	MDR
$JMP\ x: PC < - DR(x)$	PCDR
$JMP_D_1: DR < - M[AR]$	العملية موجودة مسبقاً
$JMP_D_2: PC < - DR(x)$	العملية موجودة مسبقاً
$NOP: nothing$	NOP

ملاحظة:

نسمي التعليمة حسب السجلات أو العملية المنطقية التي تتم ضمن العملية الصغرية مثلاً:

- العملية: $AR < - AC$ التسمية: ARAC $\leq$

- العملية: $AC < - AC + R$ التسمية: ADDR $\leq$

فحصل على التسميات التالية:

{ARPC, DRM, INCPC, IRDR, ARDR, RAC, ADDR, AND, RDR, DRAC, MDR, PCDR, NOP}

نلاحظ أن لدينا 13 عملية صغرية مختلفة، ويخصص لكل منها بت، وبالتالي فإن عرض السطر الكلي:

Select 1 bit	Address 5 bit	CONTROL FIELD 13 bit
-------------------------------	--------------------------------	---------------------------------------

حجم الذاكرة الصغرية = $(13 + 5 + 1) \times 17 = 323$ بت

هنا يلزم دائرة منطقية إضافية لاستنتاج إشارات التحكم

٤ تمرين 3:

ما هو حجم ذاكرة التحكم الصغرية اللازمة من أجل المسألة السابقة بفرض استخدام الترميز الشاقولي؟

الحل:

يختلف عن الطلب السابق بحجم حقل التحكم Control Field حيث يتم فيه ترميز العمليات الصغرية ثنائياً نضع لائحة بتسميات للعمليات الصغرية **بدون تكرار** (مثل الطلب السابق)

{ARPC, DRM, INCPC, IRDR, ARDR, RAC, ADDR, AND, RDR, DRAC, MDR, PCDR, NOP}

نلاحظ أن لدينا 13 عملية صغرية مختلفة، وبما أنه لدينا بعض التعليمات الصغرية مؤلفة من عمليتين صغريتين معاً، لذا يتم تقسيم العمليات الصغرية إلى مجموعتين مستقلتين:

▪ الأولى G1 تضم {INCPC, ARDR, NULL} يلزمها للترميز 2 بت

▪ الثانية G2 تضم باقي العمليات الـ 11 بالإضافة إلى حالة NULL يلزمها للترميز 4 بت

{ARPC, DRM, IRDR, RAC, ADDR, AND, RDR, DRAC, MDR, PCDR, NOP, NULL}

ملاحظة:

- عند تقسيم العمليات إلى مجموعات نضع في كل مجموعة العملية NULL
- وبالتالي فإن عرض السطر الكلي:

Select 1 bit	Address 5 bit	CONTROL FIELD 2 + 4 = 6 bit
------------------------	-------------------------	---------------------------------------

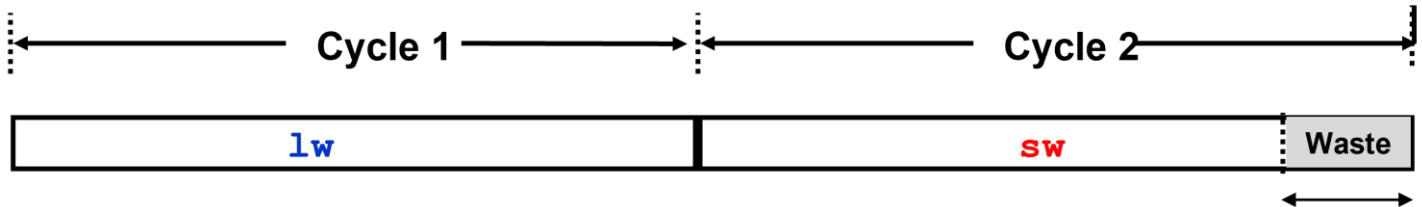
حجم الذاكرة الصغيرة = $(6 + 5 + 1) \times 17 = 204$ بت

هنا يلزم دائرة منطقية إضافية لاستنتاج إشارات التحكم

ممر المعطيات Data Path

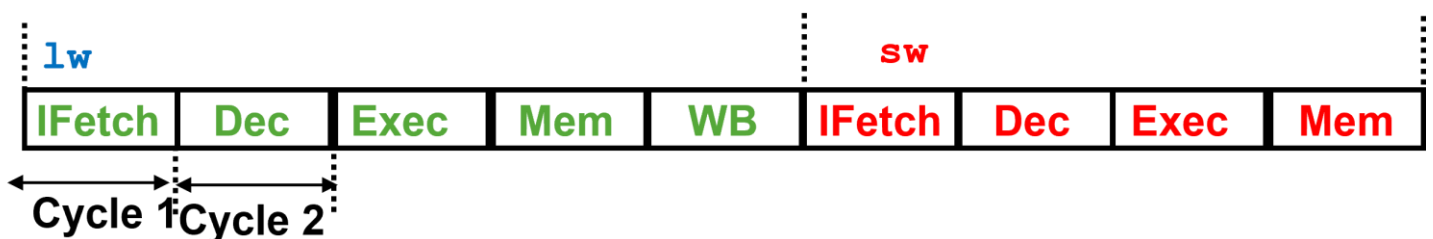
1. المسار أحادي الدور Single Cycle Data Path:

- يتم تنفيذ كامل التعليمات في دورة ساعة واحدة
- وعيوبه يتطلب دورة ساعة طويلة جداً (الوصول إلى الذاكرة أو التنفيذ) مما يقلل من تردد التشغيل



2. المسار متعدد الأدوار Multiple Cycle Data Path:

- يتم تنفيذ التعليمات إلى عدة دورات ساعة
- يمكن أن تكون دورة الساعة أقصر بكثير من SC مما يزيد من تردد التشغيل



3. المسار المتوارد Pipelined Data Path:

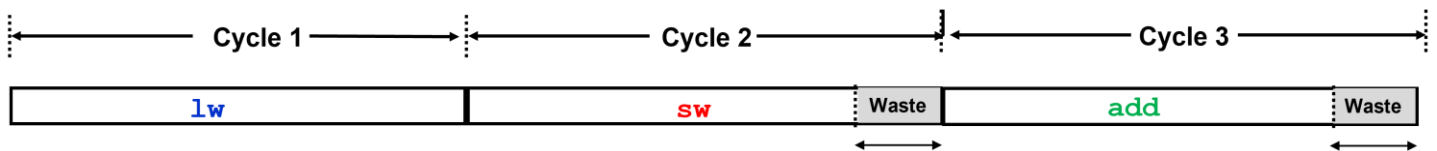
- تقسم التعليمات إلى مراحل متعددة (5 مراحل) وتنفذ تعليمات مختلفة من مراحل مختلفة بشكل متوازي. الميزة الأكبر لها أنه يتم البدء بتعليمة جديدة في كل دورة ساعة مما يزيد من إنتاجية المعالج.

تعرين:

بفرض تم تنفيذ 3 تعليمات هي lw, sw, add على ثلاثة معالجات: أحادي الدور SC فيه $T = 15ns$ ، ومتعدد الأدوار MC فيه $T = 3ns$ ، ومتوارد PL ذي 5 مراحل فيه $T = 3ns$ وبفرض عدم وجود أية أعطال، ما هو زمن التنفيذ بالحالات الثلاثة؟

1. في مسار المعطيات أحادي الدور Single Cycle Data Path:

$$CPU_{time} = CC \times T = 3 \times 15 = 45ns$$



2. في مسار المعطيات متعدد الأدوار Multiple Cycle Data Path:

$$CPU_{time} = CC \times T = (5 + 4 + 4) \times 3 = 39ns$$



3. في مسار المعطيات المتوارد Pipelined Data Path:

$$CPU_{time} = CC \times T = (5 + 2) \times 3 = 21ns$$

